School of Computer Science
Faculty of Science and Engineering
University of Nottingham
Malaysia Campus

UG FINAL YEAR DISSERTATION REPORT

Efficient Video Object Segmentation and Tracking with Intelligent Scissors, CONDENSATION, and CNN

Student's Name: Anshali Manoharan
Student Number: 20506330
Supervisor Name: Dr. Tissa Chandesa
Year: 2025

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF BACHELOR OF SCIENCE IN COMPUTER SCIENCE WITH ARTIFICIAL INTELLIGENCE (HONS)

THE UNIVERSITY OF NOTTINGHAM

Efficient Video Object Segmentation and Tracking with Intelligent Scissors, CONDENSATION and CNN

Submitted in May 2025, in partial fulfilment of the conditions of the award of the degrees B.Sc.

-Anshali Manoharan-
School of Computer Science
Faculty of Science and Engineering
University of Nottingham
Malaysia

I hereby declare that this dissertation is my own work, except where indicated in the text.

Signature: 

Date: 04/05/2025

Acknowledgement

First and foremost, I would like to express my deepest gratitude to my supervisor, Dr. Tissa Chandesa, for his invaluable guidance, support, and patience throughout this project. His expertise and encouragement have been instrumental in helping me work through the challenges of this dissertation. I am also deeply grateful to my family and friends for their unwavering support and encouragement.

Abstract

This dissertation presents a Segmentation-based Object Tracking framework (SOT). The proposed pipeline integrates Intelligent Scissors for interactive, pixel-accurate object segmentation with a lightweight implementation of the CONDENSATION algorithm for contour-based object tracking in video sequences. To evaluate tracking accuracy without relying on traditional ground truth annotations, a deep learning-based evaluation framework is introduced, employing YOLOv11 object detection outputs to approximate ground truth bounding boxes. This approach enables real-time analysis of Intersection over Union (IoU) and centre error metrics. The implementation achieves robust performance across diverse video sequences, maintaining moderately good overlap scores and low centre error under gradual object motion and cluttered backgrounds. Performance bottlenecks were mitigated by rewriting the computationally intensive components of the Intelligent Scissors algorithm in Cython, achieving a $\sim$97.5% reduction in execution time for high-resolution images. Experimental results on the PASCAL VOC 2012 dataset and real-world video sequences validate the effectiveness of the framework for rigid single object tracking applications. The study contributes a generalisable tracking approach and a robust evaluation mechanism, opening avenues for future research in adaptive tracking and contour deformation handling.

Keywords: Intelligent Scissors, CONDENSATION, contour tracking, YOLOv11, segmentation, video object tracking, real-time evaluation

Contents

1 Introduction

Computer vision literature formally defines Video Object Segmentation (VOS) as a method which consistently segments a specific object instance throughout an entire video sequence, based on a mask provided either manually or automatically in the first frame [1]. Accurately isolating and following a single object across a video sequence is critical for applications that demand precise visual understanding. Despite the progress in deep learning-based trackers, classical methods like Intelligent Scissors and probabilistic models like CONDENSATION provide valuable advantages in terms of computational efficiency, interactivity, interpretability and robustness in cluttered scenes. This dissertation implements a framework that integrates these classical techniques with modern deep learning tools, with the goal of achieving accurate and computationally efficient single-object contour tracking.

1.1 Background and Motivation

The popularity of VOST has grown due to the advances in computing power, the availability of high-quality yet affordable video cameras, and the increasing demand for automated video analysis [2]. This domain involves combining pixel-level segmentation of images with object tracking techniques to produce masks or bounding boxes for rigid or non-rigid objects in video sequences. This field is applied in applications including video summarisation, video compression, autonomous vehicles and gesture control [1].

Given the diverse range of approaches within this field, a structured taxonomy is essential for understanding the various methodologies. Although published in 2020, the comprehensive survey by Yao et al. [1] remains one of the most relevant works in this regard, providing a broad classification of VOST methods. According to this survey, VOST methods can be categorised into two broad groups – *Video Object Segmentation (VOS)* and *Segmentation-based Object Tracking (SOT)*, as shown in Figure 1 below.

From the broad categories of VOST, this study specifically focuses on the development of a *bottom-up Segmentation-based Object Tracking (SOT)* framework, as highlighted in purple in the taxonomy illustrated in Figure 1.

According to Yao et al. [1], such frameworks begin with precise pixel-level segmentation to delineate the object's contour in the first frame, often using techniques like graph cuts or mathematical morphology, and then integrate tracking algorithms such as particle filters to track the contour across frames. This bottom-up paradigm is visually illustrated in Figure 2.
Unlike classical tracking methods that typically rely on bounding boxes, SOT bottom-up approaches aim to capture the object's shape with greater fidelity. As noted by Belagiannis et al. [3], popular bounding box-based tracking approaches often suffers from drift over time due to the accumulation of irrelevant background data [3][4]. A way to avoid the accumulation of background data is to form a tight contour around the object of interest. This way, contour-based segmentation methods ensure a more faithful target representation.

This raises the question as to how well Intelligent Scissors, which is a user-assisted

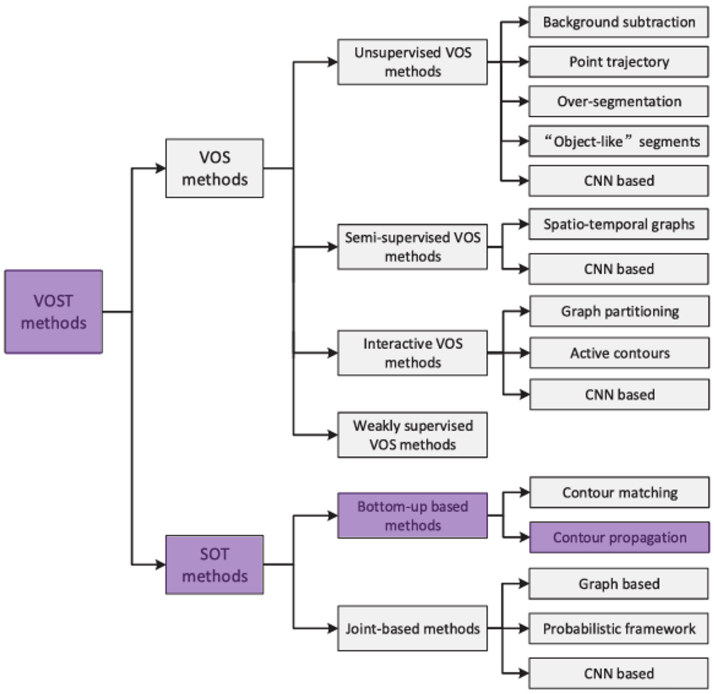


Figure 1: Taxonomy for Video Object Segmentation and Tracking (VOST) methods. The highlighted boxes represent the characteristics of the VOST method developed in this project. Source: Adapted from [1].

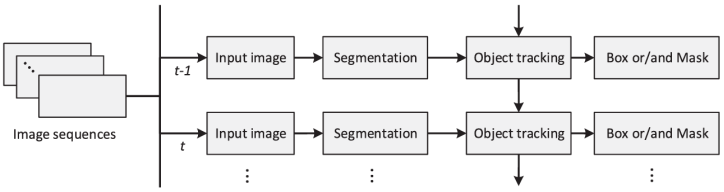


Figure 2: Bottom-up based framework. Source: Adapted from [1].

segmentation algorithm can do in this role. As of this writing, no prior research appears to have explored this exact integration of Intelligent Scissors with the CONDENSATION algorithm for contour-based video object tracking. Therefore, this study implements a SOT pipeline that begins with Intelligent Scissors for interactive, pixel-accurate segmentation, and then applies the CONDENSATION algorithm to track the object's shape across video frames with resilience to clutter and occlusion.

This motivates the following central research questions of this work:

1. *How can the object contour extracted by the Intelligent Scissors algorithm be effectively represented and incorporated within the CONDENSATION algorithm for robust contour-based tracking?*

2. *What evaluation framework can be used to assess the tracking accuracy and contour consistency of the proposed framework across video sequences?*

3. *Can the proposed contour-based tracking framework be implemented in a computationally efficient and user-friendly manner without compromising*

segmentation precision or tracking stability?

In addition, this study also adds on several algorithmic enhancements at different stages of the original segmentation and tracking frameworks from recent computer vision literature to effectively address the same set of challenges while improving overall simplicity and efficiency.

This dissertation is structured to guide the reader through the research objectives, methodology, implementation, and outcomes in a coherent and logical manner. Section 2 outlines the aims, scope, and objectives that form the foundation of the study. Section 3 presents a comprehensive review of the relevant literature, contextualizing the research within existing work. Section 4 introduces the proposed framework, detailing the development of each of the three key components—Intelligent Scissors, CONDENSATION, and (CNN)—alongside the design of the Graphical User Interface (GUI). The implementation and testing processes are described in Section 5, followed by Section 6, which outlines the evaluation setup used to assess the system's performance. Section 7 presents the results and provides a critical discussion of the findings. The dissertation concludes in Section 8. Section 9 highlights the specific contributions made by this research, while Section 10 proposes directions for future work. Finally, Section 11 offers a reflective account of the entire project, examining both its strengths and areas for improvement.

2 Project Vision

2.1 Aims

This dissertation aims to implement and evaluate the following proposed Video Object Segmentation and Tracking (VOST) bottom-up Segmentation-based Object Tracking (SOT) framework, which integrates:

- The Intelligent Scissors algorithm for target object segmentation.
- The CONDENSATION algorithm for contour-based single object tracking.

The primary objective is to assess the performance, strengths, and limitations of this approach in object contour tracking.

2.2 Scope and Objectives

The objectives of this project are structured into the following key components:

1. **Object Contour Retrieval**: Implement the Intelligent Scissors algorithm to enable efficient object contour extraction, focusing on:
    - Minimising computational costs to ensure optimal performance for user interaction.
    - Preserving segmentation accuracy despite computational optimisations.

2. **Object Tracking**: Implement the CONDENSATION algorithm to achieve object tracking, ensuring full utilisation of the algorithm's advantages, including handling clutter, while maintaining computational efficiency.

3. **Evaluation and Performance Analysis**: Embed a Convolutional Neural Network (CNN) or Vision Transformer (ViT) framework to test segmentation accuracy and tracking speed efficiently and in real-time:

    - Conduct comparative literature analysis of the accuracy and efficiency between the CNN and ViT frameworks.
    - Integrate the more suitable model (CNN or ViT) into the final version of the tracking application.

The framework is to be developed under the following constraints and assumptions:

1. The object in the video sequence is assumed to exhibit continuous and gradual motion.

2. The framework is limited to tracking rigid objects, as it relies on the initial contour from the first frame and does not accommodate significant structural changes or the emergence of new object features.

3. The framework assumes that the appearance within the initial contour remains the same throughout the entire video sequence.

4. The object's movement is restricted to the x and y directions, without consideration of rotational or depth-based motion.

5. The tracking falls under offline object tracking.

Overall, this chapter presents a foundational framework for Segmentation-based Object Tracking using Intelligent Scissors for contour segmentation and the CONDENSATION algorithm for tracking. It aims to balance accuracy and efficiency, with CNN/ViT models to aid evaluation. The system assumes rigid, gradually moving objects with consistent appearance and is currently limited to 2D offline tracking. Future improvements can address these constraints for broader applicability.

3 Literature Review

This literature review covers a range of topics in detail to inform the proposed framework. Firstly, existing frameworks is discussed. Then, the discussion is divided into three components: a review of the background, developments, and limitations of Intelligent Scissors, an analysis of the CONDENSATION algorithm's evolution, adaptations and limitations and lastly, a technical review of potential Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs) that could be integrated into the proposed framework. These related works constitute the basis for understanding the challenges and advancements that inform the design of the proposed system.

3.1 Existing SOT Frameworks

There is a large body of related approaches, however, only works that are arguably most related to the proposed framework in this study are reviewed. In the past fifteen years, segmentation-based object tracking frameworks for single object tracking have been proposed, each utilising different strategies to tackle challenges such as the weakness of bounding boxes. V. Belagiannis et al. [3] pointed how classical tracking methods rely on the bounding box to surround the target object, which quite often introduces background information. This propagates in time and its accumulation quite often results in drift and tracking failure [3][4], especially with particle filtering approach. To mitigate this, the study proposed an SOT framework - which used the GrabCut algorithm to segment the object, and the particle filter for object tracking.

Knowing that histogram matching on its own can be ineffective when the object and background share similar intensity or colour distributions, their observation model is based on both the similarity of the colour and orientation histograms of the particles, using the Bhattacharya coefficient and an exponential distribution to measure the likelihood. The authors proposed two sampling strategies – one which segments the object after every iteration of the particle filter for all particles (multiple particle filter samples segmentation), and the second which segments the particle with the highest importance weight only (single particle filter samples segmentation). The authors demonstrated that this method (with either sampling strategies) was far more effective than the standard particle filter, as it had the advantage of non-rectangular sampling.

However, while this method proved superior, it sometimes suffers from weak segmentation due to the GrabCut algorithm, which sometimes included artifacts in the segmentation for the particle. Son et al. [4] introduced an online tracking algorithm which utilised a gradient boosting decision tree to adaptively model target appearances. The algorithm creates the initial segmentation masks at the first frame using the bounding box annotation using GrabCut. Designed for non-rigid and articulated objects, the method effectively handled deformations by employing a patch-based classifier and generating segmentation masks as output. Tracking was performed using particle filtering, with target likelihood estimated through a patch-level confidence map, with the decision tree continuously updated with new data in a recursive manner.

Overall, this review explored segmentation-based object tracking (SOT) methods similar to this dissertation's proposed framework, aimed at overcoming the limitations of bounding box tracking. Early approaches, such as those by Belagiannis et al., integrated particle filters with segmentation algorithms like GrabCut to achieve more precise, non-rectangular object tracking. While this method improved robustness, it was often hindered by segmentation artifacts and reliance on handcrafted features. Subsequent work introduced adaptive appearance modeling using decision trees and patch-based classifiers to better track deformable objects. Notably, despite the common use of GrabCut in these SOT frameworks, no existing studies have investigated the integration of Intelligent Scissors with the CONDENSATION algorithm (particle filter) in segmentation-based object tracking.

3.2 Component 1: Intelligent Scissors

Background

Intelligent Scissors was first proposed by Mortensen and Barrett, an interactive tool for image segmentation and composition [5]. The tool formulates the segmentation problem as a graph search, where the image is modelled as a weighted graph, and pixels represent nodes connected by weighted edges. Optimal paths between nodes are computed in real time using Dijkstra's algorithm, where the detected edges get the least cost. The segmentation process begins with the user specifying a seed point near the target object's boundary. From this seed, the algorithm computes the lowest-cost path to every other pixel in the image. To compute the link cost between pixel nodes on the weighted graph, the sum of weighted features is calculated, which includes Laplacian zero-crossing, gradient magnitude, gradient direction, edge pixel value, inside and outside pixel value, as shown in Equation (1) later in Section 4.2.
As the user moves the cursor, the algorithm displays the optimal boundary segment between the cursor and the seed point with lowest cumulative link cost, snapping to object edges, allowing the user to outline the object of interest with ease.

The authors also considered the situation where weaker edges are desired by the user, and the shortest path follows the stronger edges instead. Dynamic training is introduced to fix this, which refines the segmentation by adjusting costs to the characteristics of the specific edge being followed. The combination of interactive feedback, graph search, and dynamic cost modelling makes Intelligent Scissors robust for segmenting desired objects from complex images.

Subsequent studies have identified a key limitation, which is the computational burden of constructing a graph for the entire image, as its size scales with the number of pixels. To address this inefficiency, various studies have proposed techniques to reduce the number of graph vertices [6-8], The next section discusses further adaptations and optimisations to improve speed and cost computation while maintaining or improving the segmentation accuracy.

Adaptations and Optimisations

Several adaptations of the Intelligent Scissors algorithm exist in medical imaging [9,10,12] and video object segmentation [11]. These adaptations come in the form of reduced feature sets, graph adjustments and new path computation strategies.

Stalling et al. proposed a graph pruning method to eliminate acute angles, resulting a 25% performance improvement [9], while Li et al. [12] introduced the Canny operator in the local cost calculation for the segmentation of vessels in coronary angiography imaging. This is not the only study to do so, Jin et al. also used the Canny operator in place of the Laplace operator in the local cost calculation, which led to an increase of 34% in IOU index for medical image segmentation [10]. Interestingly, they also used a simpler gradient direction calculation using arctan() compared to the more complex calculation proposed by the original authors [29]. This will be further discussed in the following sections, as the generalisability of this replacement to other applications needs to be explored.

For video object segmentation, Gaobo et al. suggested four modifications to Intelligent Scissors [11] to improve processing speed. One key improvement involves limiting the graph search range using a bounding box around the object of interest. The feature set was reduced to include only edge location, gradient magnitude, and Laplacian zero-crossing for local cost calculations.

To further enhance performance, an adaptive thresholding technique, referred to as a controlled flooding algorithm, was introduced to prevent graph searches from exploring non-edge regions, improving speed and robustness against noise. Finally, an on-demand path computation strategy was implemented, computing only necessary paths, replacing the full-path computation of the original method [5]. This approach stores intermediate paths for reuse. Overall, this resulted in a 6–8 times increase in processing speed with minimal loss in accuracy.

In summary, it is observed that the Canny operator has been introduced into the feature set by several studies, and the adaptations made to reduce computational costs usually included a reduced feature set in both medical imaging and video object segmentation, which not only enhanced processing speed but also maintained or even improved segmentation accuracy in specific contexts. This prompts this study to experiment with reduced feature sets.

3.3 Component 2: CONDENSATION Algorithm

Background

The Conditional Density Propagation (CONDENSATION) algorithm was first introduced by Isard et al. [13] and has since become a foundational tracking technique in the field of computer vision, influencing various applications and subsequent research.

Often interchangeably named as the particle filter [3], the algorithm is a stochastic framework which can track objects in video sequences (modelled as curves), within heavy visual clutter. The CONDENSATION algorithm overcomes the weaknesses of the Kalman filter, which cannot represent multimodal state densities (clutter) because it is based on Gaussian densities (which are unimodal).

By using 'factored sampling', the CONDENSATION algorithm tracks the contour of an object by propagating a randomly generated set of possible interpretations of the contour. Additionally, it assumes object dynamics follow a temporal Markov chain, where the new state depends only on the immediate past state. Despite its stochastic nature, it runs in near real-time by maintaining relatively tight distributions around tracked shapes.

To support detailed comparisons in the following section of existing CONDENSATION variants, a more detailed exposition of the CONDENSATION algorithm is presented here in this background section. Initially, the object contour is represented by a *state vector*, which encodes relevant parameters such as position, velocity, and other defining characteristics. A *sample set* is then generated based on this state vector, where each sample represents a potential future state vector of the object at time t+1. Each sample

is assigned an associated probability π, indicating the likelihood that the object occupies the given state.

The sample set is structured as $(s^{(n)}, \pi^{(n)}, c^{(n)})$ where $n = 1...N$ where N is the total number of samples within the set, π represents the probability of the sample, s represents the state vector, and c represents the cumulative probability. At each frame k, every sample within the sample set is updated as follows:

For the n-th sample in N samples within the sample set:

1. Use systematic sampling to pick the sample to replace the current one. The higher the probability π of a sample, the higher the likelihood that the object occupies that state, and therefore the more likely it is to be picked.

2. This newly drawn sample's state vector s is then input into a linear stochastic differential equation (also known as the state transition model) and a new state vector is generated: $s = As + Bw$, where A is the matrix representing the deterministic component of the object's dynamics, B is the stochastic component, and w is a vector of standard normal random variates.

3. Next, the probability π of this newly generated sample value is calculated by observing and using the measured features (for example, the edges of the tracked object) in the frame k.

After the sample set is recalculated, the mean value of all the samples represents the final prediction of the object in the current frame. A more visual representation of this process can be seen in Figure 3 below. The x-axis represents the sample index, while the y-axis shows the corresponding probability assigned to each sample. The diagram visualises how the sample set evolves through three key steps: (1) Systematic resampling, where samples are selected based on their probabilities from the previous frame (favoring higher-probability states); (2) Deterministic drift and stochastic diffusion, where the state vector of each sample is propagated forward using a linear stochastic differential equation - drift represents the predictable dynamics (matrix A), while diffusion introduces randomness (matrix B with Gaussian noise w), and (3) Measurement update, where the probability of each sample is re-evaluated based on how well it matches observed features (such as object contours) in the current frame. This process allows the algorithm to both track and adapt to motion and appearance changes.

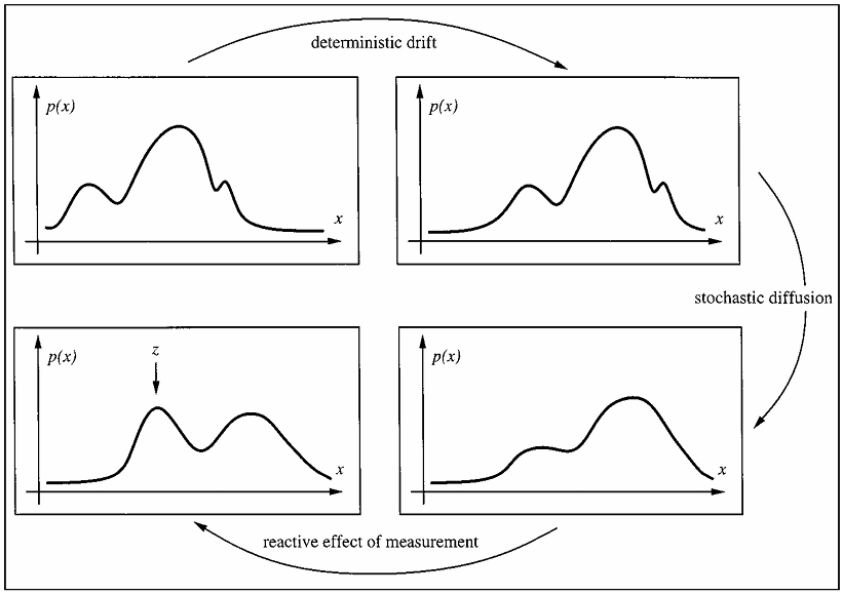


Figure 3: Illustration of one iteration in the CONDENSATION algorithm's sample set evolution. Source: paper by Isard et al. [13].

The drawback observed in this algorithm is that it relies on a predefined state transition model, typically characterised by matrices A and B (Step 2). These parameters are commonly optimised using Maximum Likelihood Estimation (MLE), or alternatively, they may be manually tuned based on input data during typical object motion [13].

However, as the size of the state vector grows, the number of parameters in matrices A and B increases quadratically. This makes manual tuning more complex and time-consuming. But recent research on adapting the CONDENSATION algorithm in industrial applications have shown that the state transition model can be redefined, and parameters can be simplified, making it easier to hand-tune.

Adaptations and Optimisations

This section of the review critically examines adaptations of the CONDENSATION algorithm across industrial applications, focusing on modifications that preserve its core framework while reducing parameter dependence and improving efficiency.

One such approach, proposed by Lui et al. [14] introduces a robust face tracking algorithm which is adaptive during execution, and requires no prior training. Unlike in the original paper [13], where the object is represented by a set of B-spline curves, the object (face) representation used here is a bounding box around the face, and they represent the state vector as six affine transformations which transform the bounding box with horizontal and vertical scaling, rotation, skew, horizontal and vertical translation. This study introduces cascaded factored sampling, which uses coarse and fine sampling into the CONDENSATION algorithm, using coarse sampling to identify the rough location of the face, and fine sampling to fine tune the face positions.

This paper also redefines the state vector using affine transformation parameters instead of traditional position and velocity. Unlike the original state transition model

[13], which relies on deterministic (A matrix) and stochastic (B matrix) components, the adapted state transition model directly applies an affine transformation to the bounding box coordinates to predict its motion across frames. Furthermore, the adaptation maintains a processing speed of 8.74 frames per second.

A different approach, proposed by Zhang et al. [15], further demonstrates how the CONDENSATION framework can be adapted, specifically for ultrasound tracking of organs. The study adapts the algorithm and tracking results show that the adapted version is capable for real-time application with processing time less than 5 milliseconds per frame and robust on low contrast target structures with a precision below 1.6 mm in translation. Interestingly, the parameters within the methodology is only a scalar A parameter to represent the weight of the propagated sample relative to the stochastic diffusion process in the model, and a sigma value for Gaussian diffusion width. Unlike the aforementioned work, which used a bounding box for tracking facial features, this study on ultrasound tracking uses coordinates to represent the object to be tracked as its state vector. This study uses a similar state representation (scaling, rotation and translation in 5 degrees of freedom). The framework allows users to manually select points on the ultrasound image frame, distinguishing between expected bright and dark regions of the object, which would then be used to calculate the observation density for the current frame. This representation of the object (bright and dark points) is not applicable to other domains, however, the simpler set of parameters and framework used will be further explored in this study.

In summary, two works deemed feasible for this dissertation's framework were critically analysed to see how the CONDENSATION algorithm has been adapted. Both works demonstrated how adaptations of different state representations, simpler state transition models and deviating from the standard methodology can achieve greater efficiency. In particular, the given frameworks demonstrated how the CONDENSATION algorithm can be adapted such that the contour provided by Intelligent Scissors can be transformed using an affine transformation-based state transition model, with simpler parameters and improved computational efficiency.

3.4 Component 3: CNNs and ViTs

The objective of this review is to critically evaluate recent advancements in computer vision literature for CNNs and ViTs to determine the most appropriate model for incorporation into the dissertation's evaluation framework.

Rationale for Task Selection

Before comparing architectures in this section, it becomes important to justify which task the CNN/ViT is to undertake in the evaluation component of the proposed framework. While instance segmentation and edge detection were considered, their suitability for evaluating tracking is limited. Edge detection models do not have compatible metrics for evaluating tracking, while current instance segmentation models still find small objects hard to detect according to recent surveys [16] [17]. This makes the edge-detection task an unreliable form of evaluation. Object detection, on the other hand, has drastically improved over the past few years in terms of speed, real-time

performance and accuracy [18], with state-of-the-art models now performing at a level suitable to serve as ground truth in the evaluation phase of the proposed framework.

Model Selection Criteria

The selection criteria include Average Precision (AP) on the popular COCO val2017 dataset, Frames Per Second (FPS), and computational requirements to run the model. AP is a standard metric in object detection, representing the average precision across all K classes. FPS quantifies the model's real-time inference capability, while computational requirements are assessed to ensure the model is suitable for deployment on resource-constrained hardware, which corresponds with the objective of developing an accessible object tracking application in Section 2.2.

ViTs in Object Detection

In recent years, the introduction of Vision Transformers (ViTs) [19] has prompted researchers to explore their potential for object detection. The pioneering work was brought by Carion et al. [20] with DETR (DEtection TRansformer), consisting of a pre-trained CNN backbone and transformer [21]. It performs on par with Faster-RCNN on the COCO dataset, doing better on larger objects in comparison, with an AP of 44.9 on the COCO val2017 dataset. However, it is computationally heavy, running at 10 FPS on a V100 NVIDIA GPU, and struggles with the detection of small objects [20].

Further versions to address these limitations were created - Deformable DETR by Zhu et al. [22] performs better than DETR with 10 times less training epochs, especially on smaller objects and has faster convergence. It has an improved AP of 46.2, however, its inference speed is 19 FPS (tested on a V100 NVIDIA GPU) - which punctuates its high computational requirements. Zhang et al. [23] presented a state-of-art end-to-end object detector DINO (DETR with Improved deNoising anchOr boxes), with best results on the COCO val2017 dataset, and a smaller model size and pre-training data size in contrast to the other models on the leaderboard. With an AP of 49.4 with 5 scales of feature maps, this model performs superiorly, but just like the aforementioned models, it requires high computational resources. Its inference speed was 24 FPS on 4 scales of feature maps and 10 FPS for 5 scales of feature maps and tested on an A100 NVIDIA GPU, which means it is still not capable of performing real-time on resource-constrained hardware.

In contrast to all these previous works, Zhao et al. [24] addressed the common limitation in all of the previous models - inference speed and high computational requirements. They introduced RT-DETR (Real-Time DEtection TRansformer) which beats the performance of the earlier versions of YOLO models in both accuracy and speed. It performs at 108 FPS with a ResNet50 backbone on a T4 GPU, with a superior AP of 53.1 on the COCO val2017 dataset, which is comparatively much better than the previous works. However, this model fails where the aforementioned models succeeded, its main limitation is its inferior performance on small objects.

In summary, while transformer-based object detectors such as DETR, Deformable DETR, and DINO have advanced object detection performance, particularly in terms of end-to-end learning and accuracy, they remain constrained by high computational costs

and limited real-time applicability, especially on resource-constrained devices. RT-DETR addresses these limitations by offering real-time performance with competitive accuracy, yet it trades off performance on small object detection.

CNNs in Object Detection

The deep learning approach for object detection first began with CNNs. Currently, popular state-of-art CNNs in object detection include the YOLO model series, R-CNN and its variants. One of the more classical object detectors that exist is the Faster R-CNN model by Ren et al. [27], which introduced RPNs (Region Proposal Networks) for the first time.

It achieved state-of-art accuracy in its time on datasets like PASCAL VOC 2007 and 2012, but ran on 5 FPS on a GPU. Another notable approach is by Tan et al., who proposed EfficientDet [25], a family of object detectors. This approach provides state-of-art accuracy while using significantly fewer parameters and computational resources compared to earlier object detection and segmentation approaches. EfficientDet-D7, the most accurate of the variants, achieves a superior AP of 51.8 on the COCO validation dataset, but the paper does not report its FPS [25]. However, its inference latency for a batch size of 1 is 24 seconds on a CPU, this suggests the model's computational heaviness.

Another influential family of models is the YOLO (You Only Look Once) series developed by Ultralytics, which has gained popularity for its real-time performance and continued improvements across versions. From the year 2015 to 2025, 12 versions of YOLO have been released, according to a recent survey by Jegham et al. [26]. The latest version of YOLO (YOLOv12) yielded an absence of progress as its complex architecture added computational overhead without offering improvements in performance, but it has been noted that YOLOv11 upholds a balance of accuracy and efficiency [26]. Interestingly, in a very recent technical comparison by Ultralytics [28] YOLO11x outperforms RT-DETRv2-x (RT-DETR version 2, extra large), with a mAP of 54.7, while still offering high inference speeds of 0.46 seconds on a CPU per frame. Conclusively, recent advances in object detection show a shift from accuracy-focused models like Faster R-CNN to more efficient architectures such as EfficientDet and YOLO. Among these, YOLOv11 stands out for balancing speed and accuracy.

3.5 Selected Model and Justification

While the most recent ViT (RT-DETR) provides real-time inference speeds and state-of-art mAP scores on its larger variants, the only drawback is its requirement to be deployed on competent hardware like NVIDIA T4 GPUs [28], which does not suit the constraints for this dissertation's proposed framework. In contrast, the YOLO version 11 of models performs on par with the variants of RT-DETR, while accentuating fast inference with minimal accuracy trade-offs. Therefore, based on the findings of this technical review, the YOLOv11 models have been selected for further analysis and integration into the proposed framework of this dissertation.

Conclusively, this chapter explored the existing SOT frameworks, the adaptations and

optimisations for each component (Intelligent Scissors, CONDENSATION and CNN/ViTs). With each section, possible ways of increasing the efficiency of the framework were introduced, and these will inform the development of the proposed framework in Section 4.

4 Proposed Framework

This section discusses the proposed framework for the GUI, Intelligent Scissors, CONDENSATION and the YOLOv11 in detail, based on the insights drawn from the literature review. It includes justifications for modifications to the original frameworks, along with the rationale behind selected hyper parameters and other implementation choices.

4.1 Overview - GUI and Overall Functionality

The UI is kept simple, prioritising the functionality of the underlying algorithm. The workflow is outlined below:

Stage 1: Intelligent Scissors

(a) The user selects a video file through a file dialog interface.

(b) The first frame of the selected video is rendered for user interaction.

(c) The user places a seed at a desired position on the image. Multiple seed placements are allowed to test and refine the positioning, with the cursor snap feature ensuring the seed snaps to the nearest edge boundary based on edge magnitude.

(d) Once satisfied, the user confirms the seed placement by pressing the spacebar.

(e) The algorithm expands nodes from the confirmed seed point.

(f) The user can toggle the 'd' key to enable and disable dynamic training when working with weaker edges.

(g) Now highlighted paths from the seed to the current cursor position are displayed, allowing the user to evaluate possible contours.

(h) Steps (c) to (g) are repeated as needed for outlining the entire object contour.

(i) When satisfied with the segmentation, the user presses Enter to finalise the object contour.

Stage 2: CONDENSATION Algorithm

Since no user interaction is required during the execution of this phase, the GUI remains minimalistic. The current frame number will be displayed at the top of the window, and the video is to be played in real time while the CONDENSATION algorithm continuously performs object tracking throughout the sequence.

Stage 3: Evaluation with YOLOv11

The chosen object detection model YOLOv11 will provide ground truth annotations for each frame, which are rendered with red bounding boxes, while the tracking result from the CONDENSATION algorithm should be visualised using green bounding boxes. The Intersection over Union (IoU) between the predicted and ground truth boxes will be computed and displayed in real time in the top-left corner of the window for each frame.

Upon completion of the video sequence, a plot showing the IoU values across all frames will be presented, accompanied by the centre error displayed in the bottom-left corner of the figure.

4.2 Component 1: Intelligent Scissors

This section outlines the proposed framework for Intelligent Scissors, informed by the literature review in Section 3.2.

Graph Construction and Node Representation

Firstly, a graph structure models the image as a weighted graph, where each pixel is a node, and edges represent neighbouring relationships. The graph is constructed with each pixel is encapsulated as a Node object containing attributes like its coordinate location, cumulative cost, a pointer to the previous node, and an expansion flag (expanded). The pointer determines the direction of the edge, while the cumulative cost determines the cost of the edge.

Local Cost Calculation and Weights

The detailed paper for Intelligent Scissors by the original authors Mortensen et al. [29] suggested the following local cost calculation during training for the edge between two nodes p and q:

$$l(\mathbf{p},\mathbf{q}) = w_Z \cdot f_Z(\mathbf{q}) + w_G \cdot f_G(\mathbf{q}) + w_D \cdot f_D(\mathbf{p},\mathbf{q}) + w_P \cdot f_P(\mathbf{q}) + w_I \cdot f_I(\mathbf{q}) + w_O \cdot f_O(\mathbf{q}) \quad (1)$$

where each term is explained by Table 1. below.

Table 1: Formulations of Image Features Used in Segmentation

Image Feature	**Formulation**
Laplacian Zero Crossing	f_Z
Gradient Magnitude	f_G
Gradient Direction	f_D
Edge Pixel Value	f_P
'Inside' Pixel Value	f_I
'Outside' Pixel Value	f_O

The weights w in Equation (1) represents the weighted importance for each feature within the equation. However, Gaobo et al. [11] demonstrated that the most useful features are the edge location, gradient magnitude and Laplacian zero-crossings. Hence, the cost function in Equation (1) can be simplified as:

$$l(\mathbf{p}, \mathbf{q}) = w_Z \cdot f_Z(\mathbf{q}) + w_G \cdot f_G(\mathbf{q}) + w_D \cdot f_D(\mathbf{p}, \mathbf{q}) \tag{2}$$

Many studies also discussed the use of Canny operator instead of the Laplacian zero crossing operator [10][12], suggesting that the Canny operator lessens the generation of pseudo edges and preserves weaker edges, enhancing the original segmentation effect. Based on these suggestions, the following link cost function is derived:

$$l(\mathbf{p}, \mathbf{q}) = w_C \cdot f_C(\mathbf{q}) + w_G \cdot f_G(\mathbf{q}) + w_D \cdot f_D(\mathbf{p}, \mathbf{q}) \tag{3}$$

where $f_C(q)$ represents the Canny operator. According to Gaobo et al. [11], the weights w_Z, w_G, and w_D are set to 0.43, 0.14, and 0.43, respectively—these values were found to perform well across a range of video sequences. However, in this framework's proposed approach, where the Laplace operator is replaced by the Canny edge detector, the weightage has to be reconsidered. To reflect the improved reliability of Canny and the increased relevance of gradient magnitude, the weights have been adjusted to 0.4, 0.3, and 0.3 for w_C, w_G, and w_D, respectively.

This re-weighting also aligns with the use of Canny and gradient magnitude in dynamic training (which will be detailed in a later subsection), where giving them greater influence is essential for accurately capturing weaker edges. Lastly, the Canny and gradient magnitude terms are directly computed from the filters applied on the first frame of the video, but gradient direction f_D is to be calculated by the following equation, as adapted from the work by Jin et al.[10]:

$$\theta_D = \arctan\left(\frac{g_x}{g_y}\right) \tag{4}$$

where g_x and g_y are gradient direction calculated using Sobel operators in the x and y directions respectively.

Cursor Snap Feature

Planting seeds directly on image boundaries can be challenging and time consuming for users. To address this issue, the original authors [29] introduced the cursor snap technique. This method ensures that a planted seed automatically snaps to the nearest image boundary by evaluating the edge magnitude within a p $\times$ p neighbourhood around the cursor. A neighbour hood size of 7 $\times$ 7 was selected, as it generalises well for all images and aligns with the range specified in the original paper [29].

Path-Finding Algorithm

The path-finding algorithm finds the shortest path from the seed to the user's hover point real time, which requires calculating the shortest paths to the seed from all the pixels of the image. To do this, the Dijkstra's algorithm for graph search, is used by employing an O(N) bucket sorting mechanism for managing the active list, where N

denotes the total number of pixels in the image [29]. This optimisation parallels Dial's implementation of Dijkstra's algorithm, which also uses bucket sorting to efficiently organise and process nodes with similar cost values.

Dynamic Training

The dynamic training process is also adopted in this framework, by executing the following steps:

(a) Compute Canny and gradient magnitude from the grayscale image (first frame of the video).

(b) User selects at least two seeds and confirms a contour segment.

(c) Use the last N pixels (eg. 512) from the confirmed contour segment and sample the feature values from the precomputed Canny and gradient magnitude images.

(d) Build histograms from these feature values. Invert and normalise these histograms so that the frequently occurring features in the contour segment receive lower costs.

(e) During contour path finding, use these trained cost maps to calculate link costs.

(f) As the user confirms new path segments, reiterate Step (c) onwards, recalculating the training histograms to adapt to the new features in the newest confirmed path segment.

4.3 Component 2: CONDENSATION Algorithm

The following proposed framework for the CONDENSATION algorithm is a simple variant developed by referring to the works described in the literature review in Section 3.3.

State Vector Representation

In the work by Lui et al. [14], they use a state vector with six degrees of freedom *[horizontal scaling, vertical scaling, rotated angle, skew, horizontal translation, vertical translation]*. The state vector in this work is simply *[horizontal translation, vertical translation]* to abide by the constraints set in Section 2.2. The contour provided by Intelligent Scissors will be transformed according to this state vector representation.

State Transition Model

The state transition model used here is based on the CONDENSATION algorithm adapted for ultrasound tracking by Zhang et al. [15]:

$$s_{k+1}^{(n)} = \overline{s_k} + A \cdot \left(s_k^{\prime(n)} - \overline{s_k} \right) + \sigma \cdot G(0,1),\ n \in \{1, \ldots, N\}. \tag{5}$$

Each sample s_k from the sample set in frame k is updated using the transition model shown in Equation (5), where s_{k+1} denotes the transformed sample, and k is frame number. The term $\bar{s}_k$ r represents the mean state vector computed across the entire

sample set, and G(0,1) denotes a zero-mean Gaussian diffusion term with width σ. The term s'_k refers to the sample selected through systematic resampling. The variable n corresponds to the sample number and A is the process scaling factor that determines the relative influence of the prior state versus stochastic diffusion during particle propagation.

Algorithm

a) Initialisation: A sample set $s_0^{(n)} = \{\}$ is first generated randomly. Each sample within the sample set is assigned an equal probability p, forming a uniform prior distribution over the state space. Each sample contains a state vector *[horizontal translation, vertical translation]* which is generated randomly to represent the possible positions of the object in the next frame. Key system parameters like Gaussian width, A, and the number of samples are all initialised as well.

b) Propagation: During propagation, each sample's state vector is stochastically diffused with the state transition model in Equation (5).

c) Observation: Each propagated sample within the sample set, representing a candidate object state, is evaluated against the current video frame using a probabilistic observation model. Overall, this involves transforming the reference contour according to the state vector and measuring its alignment with the current frame features. In the original paper by Isard et al. [13], the observation model is as follows:

$$p(\mathbf{z} \mid \mathbf{x}) \propto \exp\left(-\frac{(\text{distance to edge})^2}{2\sigma^2}\right) \tag{6}$$

The observation probability of the observation z given the state x equals the Gaussian of the distance to the nearest high-contrast feature ('distance to edge') summed up over the contour. The probability p for the sample x is then finalised by normalising the calculated observation probability – by dividing by the sum of the observation probabilities for the current sample set. The cumulative probability for the sample is updated as well.

d) Resampling: After calculating the probability for each sample within the sample set, systematic resampling is performed over the entire sample set. This updates the sample set and picks the samples which are more likely to be the actual object's state in the frame k.

e) Final Contour Calculation: According to the original authors [13], the final contour for each frame is determined by computing the weighted mean of the entire set of samples. However, observations during the implementation of this framework revealed that this approach tends to accumulate errors present in other samples. As a result, an alternative method was adopted: selecting the most probable contour. Specifically, the sample with the highest probability p is chosen as the final contour for each frame. This has also been adopted in other existing studies as well [3][30].

Parameter Settings

Sample size N is set to 75. The coefficient A is set to 0.4, following the configuration in the study by Zhang et al.[15]. Lastly, the Gaussian width parameter σ, was hand-tuned and set to 1.5 based on trial and error during performance observations, as this value consistently yielded optimal tracking results.

Practical Considerations

To address occasional low-quality predictions by the CONDENSATION algorithm, a refinement step was introduced in each iteration: the 10 samples with the lowest observation probabilities are replaced with newly generated random samples. This injection of diversity into the sample set helps prevent stagnation and increases the likelihood of recovering accurate tracking. A new parameter, translation range, is introduced to control the spread of these random samples. For high-velocity objects, a larger translation range allows broader exploration and improves recovery from tracking failure. Conversely, for slower-moving or cluttered scenes, a smaller range helps maintain precision by focusing on more probable state spaces.

4.4 Component 3: Tracking Performance Evaluation

Object Detection Model for Tracking Evaluation: YOLOv11n

As per the technical literature review in Section 3.4, YOLOv11 will be used for evaluating the tracking performed by the CONDENSATION algorithm. From nano to extra-large, YOLOv11 models scale its usability from edge devices to high-performance systems. The nano model YOLO11n is chosen for this proposed framework, as it delivers considerable speed and efficiency gains over its predecessor, making it well-suited for real-time use [31].

Evaluation Metrics Used

The evaluation metrics employed in this framework are commonly found in single object tracking [32]. The first is Center Error, which measures tracking accuracy by computing the average Euclidean distance between the predicted and ground truth target centres for each frame. This metric remains popular due to its simplicity, providing a single error value per frame. The second metric is Intersection over Union (IoU), also known as the overlap score. IoU is calculated per frame and quantifies the spatial overlap between the predicted and ground truth bounding boxes. It is defined as the ratio of the area of intersection to the area of union between the two boxes.

Evaluation Framework

The evaluation of the CONDENSATION algorithm's performance will be compared to the YOLO11n model's object detections per frame, which represents ground truth. The process will begin with the user selecting a video file and annotating the target object in the first frame using Intelligent Scissors. The first frame will then be fed into YOLO11n, and the detections of YOLO11n will be compared to the annotated contour by the user. If a match is found, the corresponding object ID will be stored, and

evaluation mode is activated.

The system will track the object across subsequent frames using CONDENSATION, storing the predicted contour for each frame. In evaluation mode, the entire video will be processed by YOLO11n to obtain frame-wise detections. For each frame, the predicted contour from CONDENSATION will be matched to the closest YOLOv11 detection with the same object ID, and evaluation metrics including Center Error and Intersection over Union (IoU) will be computed.

Upon video completion, IoU will be plotted against frame number and the final Center Error will be reported. Lastly, if no initial match is found to the annotated object by the user in the first frame, the pipeline will proceed without evaluation mode, performing only tracking. The following flowchart in Figure 4. summarises the evaluation framework.

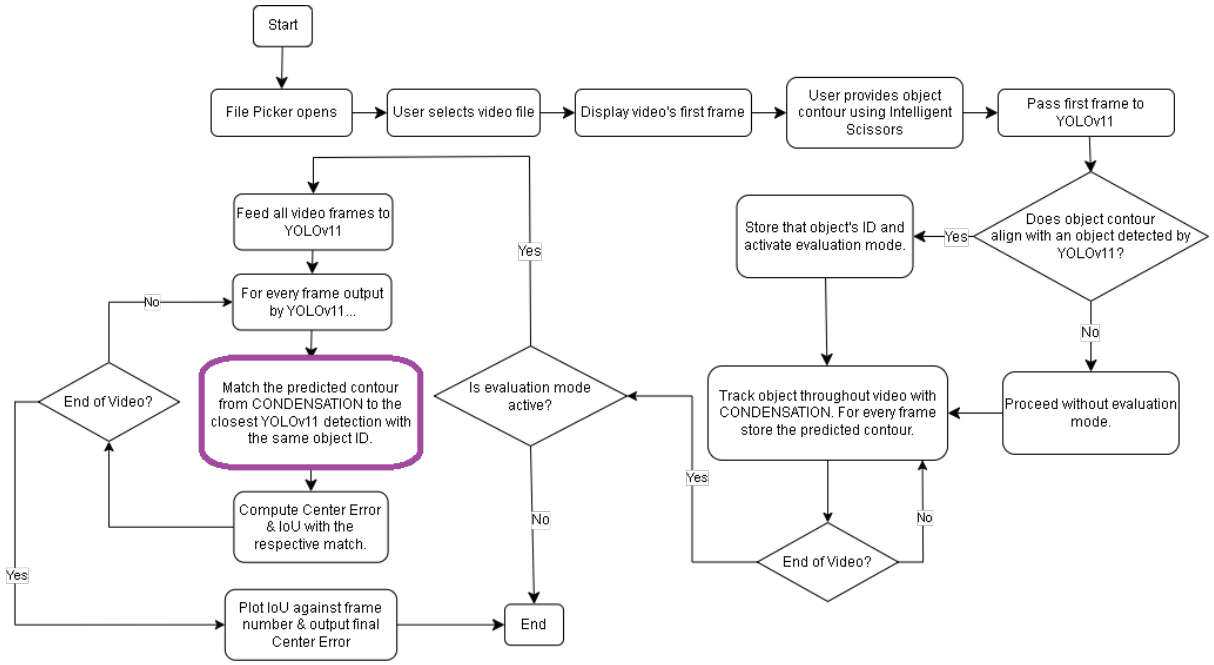


Figure 4: Evaluation framework using the YOLOv11 model.

In Figure 4, the step outlined in purple requires further calculations. To compare the CONDENSATION algorithm's predicted pixel-level contour with the YOLO11n bounding box, CONDENSATION's predicted contour should first be converted into a bounding box using the calculation described below.

Given a set of n coordinates forming the predicted contour by CONDENSATION:

$$\{(x_1, y_1), (x_2, y_2), \cdots, (x_n, y_n)\} \tag{7}$$

The bounding box coordinates B are calculated as follows:

$$x_{\min} = \min(x_1, x_2, \ldots, x_n) \tag{8}$$

$$y_{\min} = \min(y_1, y_2, \ldots, y_n) \tag{9}$$

$$x_{\max} = \max(x_1, x_2, \ldots, x_n) \tag{10}$$

$$y_{\max} = \max(y_1, y_2, \ldots, y_n) \tag{11}$$

Which results in:

$$B = [(x_{\min}, y_{\min}), (x_{\max}, y_{\min}), (x_{\max}, y_{\max}), (x_{\min}, y_{\max})] \tag{12}$$

Equations (8-11) calculate and result in the bounding box coordinates B (Equation 12) of the contour produced by the CONDENSATION algorithm. This enables the computation of evaluation metrics such as Intersection over Union (IoU) and centre error in comparison to the YOLO11n's bounding box detections.

Additionally, there is an extra note to add to this evaluation setup; the diverse video sequences used to evaluate the proposed framework also involve face tracking (Section 6), therefore, the YOLO11n model is replaced with its face-detection variant, `YOLO11n-face` for that experiment. This substitution is necessary as the original YOLOv11 model is trained to detect the 'person' class rather than faces specifically.

As a whole, this section outlined the complete design and implementation of the proposed SOT framework, integrating a user-guided Intelligent Scissors segmentation tool, the CONDENSATION algorithm for contour-based tracking, and an evaluation pipeline aided by the nano model YOLO11n. Key enhancements include a reduced feature set in Intelligent Scissors to improve efficiency, along with cursor snap and dynamic training to aid user interaction. For CONDENSATION, a simplified variant is formed by adapting from the works of Zhang et al.[15] and Lui et al.[14] discussed in Section 3.3. Performance evaluation is conducted using IoU and center error metrics, with automated comparisons against YOLO11n detections. The framework prioritises modularity and practicality, with justified parameter choices and error-handling mechanisms. Together, these components establish an efficient, interactive, and evaluative object tracking system ready for practical testing.

5 Implementation and Testing

This section outlines the libraries used in the development of the application, describes the technical challenges encountered during implementation, and explains the solutions applied. Next, a comprehensive walkthrough of the application's features is given, and finally, the testing documentation is presented.

5.1 Libraries and Modules Used

The application uses a combination of standard Python libraries, third-party packages, and custom modules to achieve its functionality. Below is a breakdown of the key libraries imported and their respective roles:

- **Standard Libraries:**
  - `time`, `math`, `heapq`: Used for time tracking, mathematical operations, and efficient data handling through heap queues for Intelligent Scissors
  - `tkinter`, `tkinter.filedialog`: Provides a graphical user interface (GUI) for file selection and user interaction.
- **Image Processing and Visualisation:**
  - `cv2` (OpenCV): Handles image loading, display, and basic image processing operations such as gradient magnitude and direction calculations.
  - `matplotlib.pyplot`: Used for plotting and visualising image data and algorithm outputs.
  - `skimage.draw.line`: Assists with enclosing incomplete contours drawn by user during Intelligent Scissors.
- **Numerical and Scientific Computing:**
  - `numpy`: Core library for numerical operations and array manipulation; extensively used throughout the application.
  - `libc.math`, `libc.stdlib` (via Cython): Provides access to C-level mathematical functions and memory management for performance-critical code.
- **Object Detection:**
  - `ultralytics.YOLO`: Employs a YOLOv11-based model for object detection (used in evaluation framework).
- **Cython and Compilation Tools:**
  - `Cython`, `setuptools`, `Extension`: Used to compile performance-sensitive parts of the application, allowing integration of C-level operations for speed optimization.
  - `cimport` and `cythonize`: Facilitate type declarations and compilation of Cython modules.

5.2 Problems Faced and Resolutions

For computationally intensive operations like the node expansion phase in the Intelligent Scissors algorithm, the execution time increased significantly with higher image resolutions when implemented in pure Python, as shown in Figure 5. For a 1080p resolution image, the pure Python implementation takes 464 seconds (almost 7 minutes).
Consequently, the Intelligent Scissors algorithm was rewritten in Cython, a superset of Python designed to facilitate C-level performance by allowing the inclusion of static type declarations and direct calls to C libraries. This optimisation enabled substantial reductions in execution time, particularly during performance-critical stages such as contour cost computation and node expansion. For lower resolution images, the process

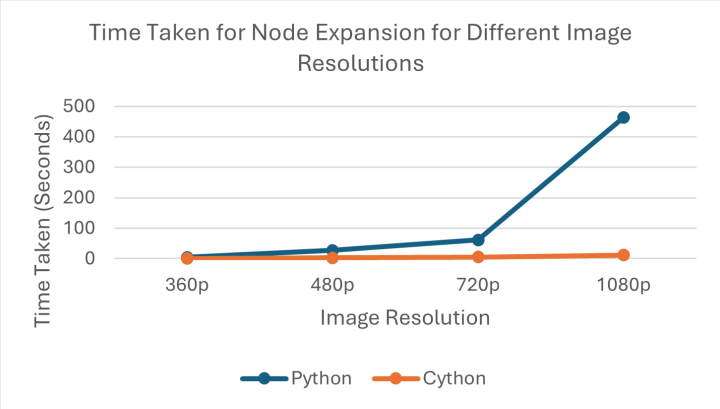
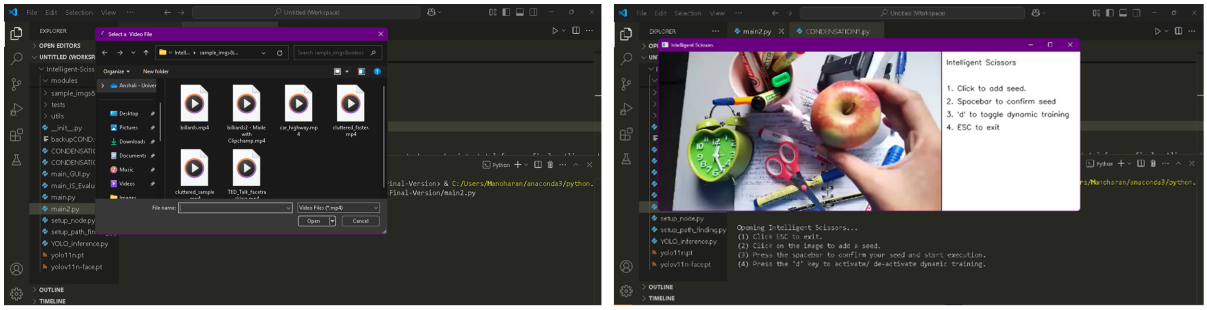


Figure 5: Time taken to expand the nodes for an image when a seed is confirmed by the user.

is almost instantaneous, and for the highest resolution image (1080p, as shown in Figure 5) it takes only around 12 seconds, which is a reduction of ∼97.5% in execution time.

5.3 Application Demonstration

The following Figures 6, 7, 8, 9, 10, 11 and 12 display the GUI in action for all three components.



(a) (b)

Figure 6: (a) File picker appears, and in (b) the chosen video's first frame is displayed for outlining the object contour using Intelligent Scissors. The instructions are dictated on a side panel.

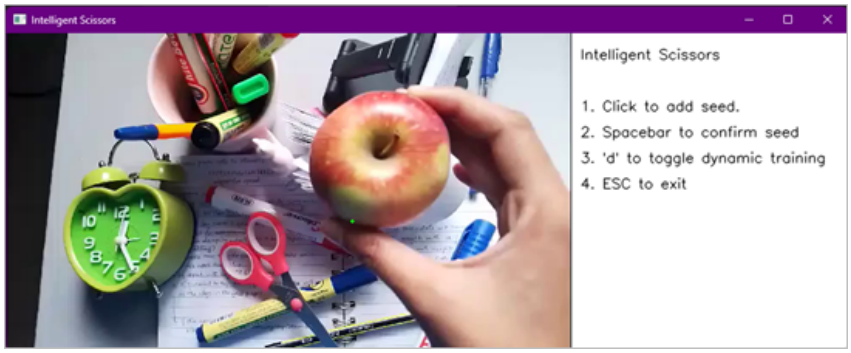
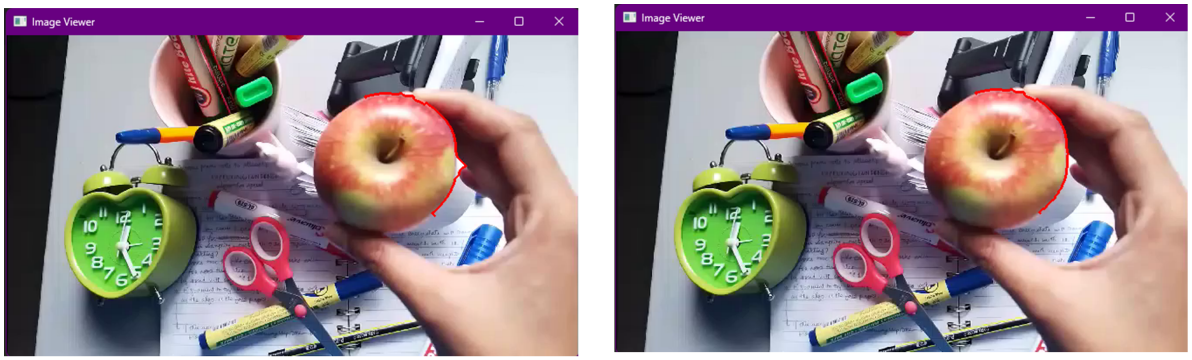


Figure 7: The user is allowed to click and experiment with seed placements (in green). The cursor snap feature allows the user to place the seed exactly on the closest edge.



(a) (b)

Figure 8: The dynamic training feature can be toggled with the 'd' key. When dynamic training mode is enabled, the weaker edges are given more priority. The difference is visible in the above images - (a) shows the line segment attracted to the stronger edge of the hand. (b) shows when dynamic training is on, the line segment learns to remain on the weak edge of the apple.

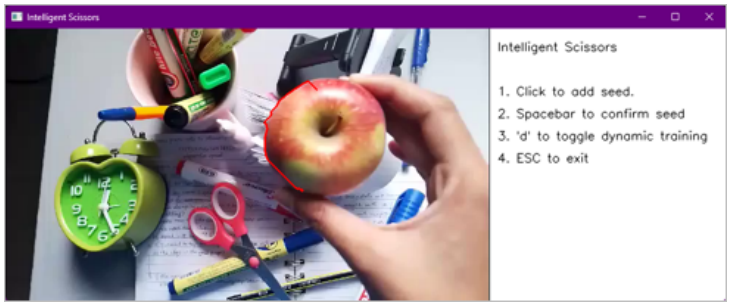


Figure 9: After a seed is confirmed, the user can hover over the image and the red line marks the shortest path to the cursor.

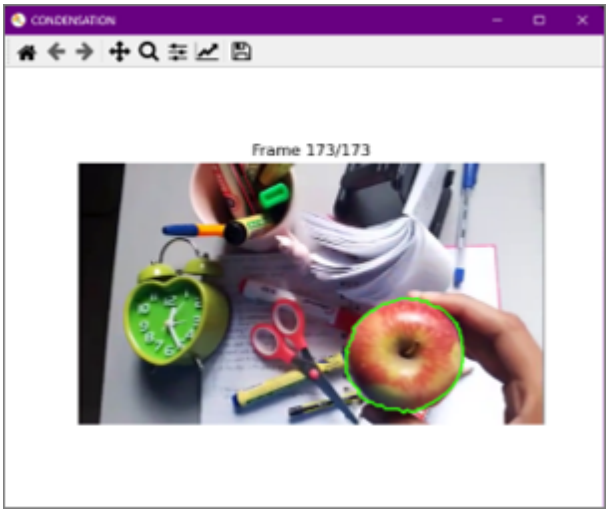


Figure 10: This window runs after the user confirms the object contour with the 'Enter' key. The CONDENSATION algorithm performs object tracking and the green outline shows the final predicted contour for that frame.

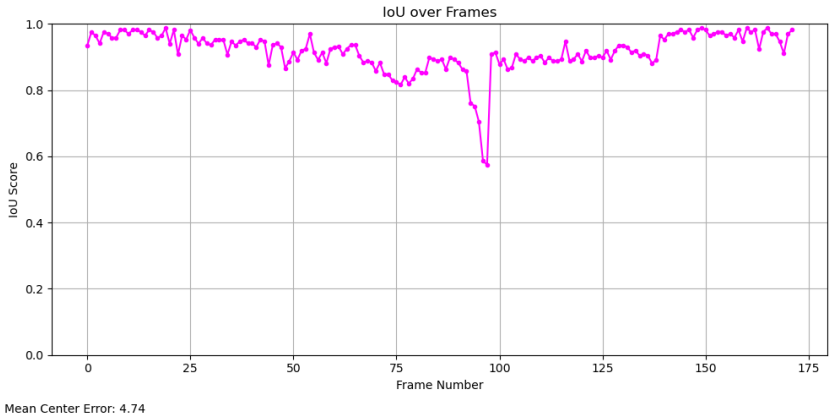


Figure 11: After the evaluation phase is over, the evaluation results (IoU over all frames and mean center error) is displayed.

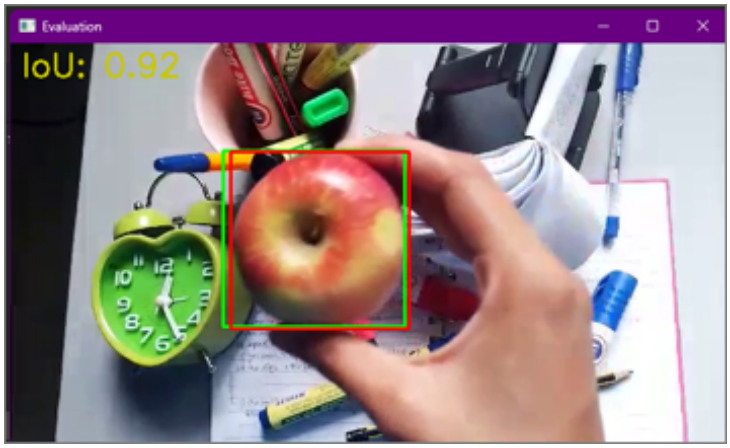


Figure 12: After the CONDENSATION algorithm's object tracking is over, the evaluation phase begins. The green bounding box is the predictions made by the CONDENSATION algorithm, and the red bounding box is the ground truth provided by YOLOv11. The IoU of the current prediction is also displayed (0.92)

Importantly, Figure 12 shows where Equations (8-12) are applied actively - the bounding box coordinates are calculated iteratively per frame for the CONDENSATION contours before being compared to YOLO11n's bounding boxes.

Lastly, the results presented in Figures 10, 11, and 12 collectively demonstrate both the effectiveness of the CONDENSATION tracking implementation and the reliability of the evaluation technique used. In Figure 10, the green contour output by the CONDENSATION algorithm clearly and consistently follows the object of interest, confirming accurate visual tracking. This qualitative assessment is reinforced by Figure 11, where the corresponding IoU and center error values remain high and stable throughout the sequence, reflecting close contour alignment in measurable terms. Crucially, Figure 12 shows a screenshot from the evaluation pipeline using YOLOv11 detections as surrogate ground truths. The near-perfect overlap between the YOLOv11 bounding box and the CONDENSATION prediction, with an IoU of 0.92 shown in real-time, provides strong visual and numerical evidence of accurate tracking. This close visual agreement validates not only the tracking results but also confirms that the evaluation framework itself is reliable—accurately quantifying what is visually evident, thereby justifying its use in place of manually annotated ground truth.

5.4 GUI Functionality Tests

This section presents the testing procedures undertaken to evaluate the functionality, usability, and performance of the developed object tracking application. Each module was tested independently as well as in combination within the graphical user interface (GUI) to ensure correct behaviour, responsiveness, and robustness.

The software application was tested on a consumer-grade laptop, with the system running Microsoft Windows 11 Home (Version 10.0.26100) with an Intel Core i5-1035G1 processor (4 cores, 8 threads) and 4 GB of RAM. This setup represents a modest hardware environment, suitable for evaluating the application's performance,

responsiveness, and stability under real-world constraints. Table 2. on the following page documents the test cases carried out on the GUI.

Bugs Identified

When the user hovers over the instruction panel, an Index Error appears as the hover point is not over the image anymore.

Bugs Resolution Summary

The identified issue was successfully resolved during the testing and debugging process: A bounds check was added to ensure the hover point is processed only when within the image area. This prevents indexing errors when the cursor moves over the instruction panel.

Performance and Stability

The Intelligent Scissors tool processes and expands all nodes across a standard-resolution image in approximately 0.5 to 1 second. The CONDENSATION algorithm operates at a frame rate of 3 to 5 FPS, ensuring responsive performance. The evaluation stage performs at approximately 8 FPS. Application startup takes roughly 5 to 10 seconds.

Overall, this section detailed the practical implementation of the proposed framework, highlighting the libraries, performance optimisations, and GUI features that underpin its functionality. Major improvements, such as Cython-based acceleration for Intelligent Scissors, significantly reduced computation time—achieving up to 97.5% faster execution on high-resolution images. The GUI integrates all components smoothly, and provides intuitive interaction and real-time feedback. Functional testing confirmed the reliability and responsiveness of each module on modest hardware, while identified bugs were successfully addressed. Overall, the implementation demonstrates robustness, efficiency, and usability, validating the framework's readiness for further evaluation.

Table 2: Test Cases for GUI

ID	Feature	Steps to Reproduce	Expected Outcome	Result
1	Load Video File	1. Run the application. 2. Select an .mp4 file.	The first frame of the video is displayed in the IS GUI window. Instruction panel must appear beside the image.	Pass
2	Cursor Snap	1. Click near a strong edge in the image. 2. Observe the snapped position printed in console.	The snapped point is close to the edge and differs slightly from the original click.	Pass
3	Node Expansion	1. Place seed by clicking. 2. Press spacebar.	After a few seconds, path appears red as user hovers over the image, seed is drawn, and "Updating nodes..." is printed.	Pass
4	Dynamic Training	1. Press the 'd' key during GUI use after forming a contour segment.	"Dynamic Training set to: True/False" message appears as expected.	Pass
5	Finalising Contour	1. Add two or more seeds and paths. 2. Press Enter.	"Object outline complete." is printed, and program proceeds to CONDENSATION.	Pass
6	CONDENSATION	1. Outline an object in the first frame of a video using Intelligent Scissors and click Enter.	"Stage 2: CONDENSATION Algorithm" is printed and entire video is played real-time with the drawn contour tracking the object.	Pass
7	CNN Evaluation	1. Complete outline of object using Intelligent Scissors, ensure the message 'Object detected by YOLOv11' appears and wait for CONDENSATION section to complete.	CNN evaluation metrics appear real-time on screen, with tracking completing without errors, outputting frames and bounding boxes for both trackers.	Pass
8	Disabled CNN Evaluation	1. Complete outline of an ambiguous object on the first frame of video and ensure the message 'Object not detected by YOLOv11.' appears.	After the CONDENSATION stage, the program must halt without any errors.	Pass

6 Evaluation

6.1 Component 1: Intelligent Scissors

This section discusses the evaluation setup and metrics to validate the performance of the implemented Intelligent Scissors framework.

- **Dataset:** For the evaluation of segmentation accuracy, the implementation of Intelligent Scissors will be tested on a subset of the PASCAL Visual Object Classes (VOC) 2012 dataset [33], which is widely recognised for benchmarking

image segmentation tasks. The dataset provides annotated images with pixel-level ground truth, enabling quantitative assessment of the algorithm's performance. A subset of 10 images chosen will include a range of object types and boundary complexities to comprehensively test the algorithm's robustness and effectiveness, as shown in the samples below in Figure 13.

Figure 13: Sample images from the PASCAL VOC 2012 dataset.

- **Evaluation Metrics:** The following evaluation metrics will be utilised to measure the accuracy and precision of the segmentation results compared to the ground truth annotations:
  - **SSIM (Structural Similarity Index Measure):** SSIM evaluates the perceptual similarity between two images by comparing their luminance, contrast, and structural information. It ranges from -1 to 1, with values closer to 1 indicating higher similarity. This metric is sensitive to visual quality and helps assess how closely the segmented contours preserve the structure of the annotated regions.
  - **Feature Similarity Index Measure (FSIM):** FSIM assesses image similarity by focusing on low-level image features, such as phase congruency and gradient magnitude. These features reflect important structural information in an image. FSIM is especially suited for detecting finer details in complex regions, with scores closer to 1 indicating better agreement.
  - **Mean Squared Error (MSE):** MSE calculates the average squared difference between the pixel intensities of the segmented output and the ground truth. A lower MSE indicates fewer discrepancies, making it an effective measure of pixel-wise accuracy, though it is less perceptually aware compared to SSIM and FSIM.
- **Ground Truth Processing:** Additionally, for practicality and consistency, only the main object per image is considered for segmentation, as shown in Figure 14 below. Focusing on a single primary object per image allows for more controlled and meaningful comparisons between algorithm performance and the ground truth annotations, minimising potential ambiguities from secondary or overlapping objects. This approach ensures the evaluation process remains robust and reliable, especially in complex images where multiple objects might introduce unnecessary variability.

Figure 14: The above images show the (a) before and (b) after of ground truth processing. Only the main object remains for evaluation in the ground truth images.

6.2 Component 2: CONDENSATION Algorithm

The performance of the implemented CONDENSATION framework will be evaluated on a set of diverse video sequences (as illustrated in Figure 15 below) to assess both its robustness and accuracy.

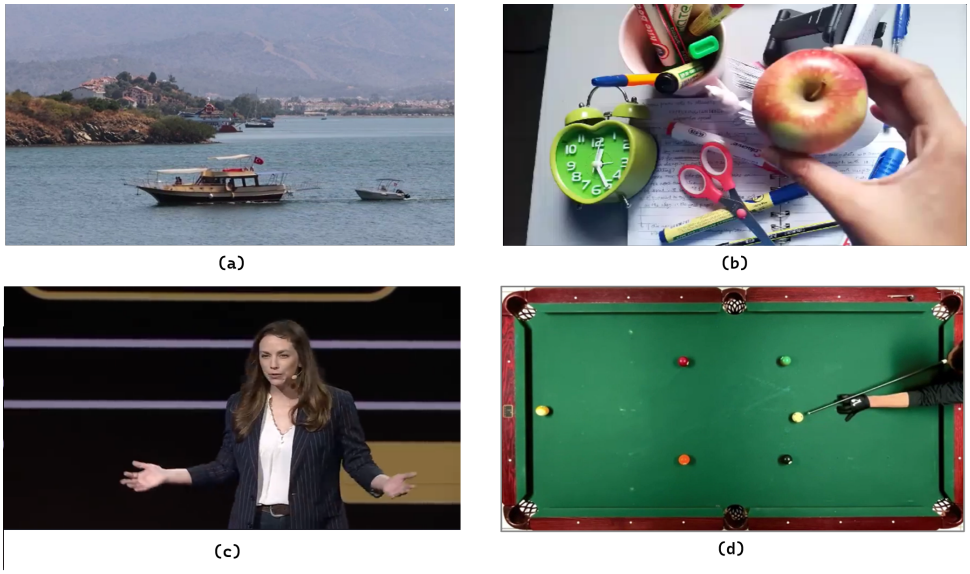


Figure 15: (a) boat in sea (b) apple in a cluttered environment (Captured by author) (c) TED Talk speaker moving while talking (d) billiards drill. Video references: Boat at sea, TED Talk speaker, Billiards drill.

Based on the motion characteristics of the object of interest in each video, the Gaussian diffusion term and translation range parameters are set according to the object dynamics. The parameters set for each video is in the following table:

Table 3: Gaussian Diffusion and Translation Parameters for each video

Video ID	Video	Gaussian Diffusion Parameter	Translation Range
1	Boat in sea	7	150
2	Apple in Clutter	5	70
3	TED Talk speaker	1	30
4	Billiards	7	150

For each sequence, the object of interest (boat, apple, speaker's face and white billiard ball) is manually outlined by the user using Intelligent Scissors. The resulting object tracks produced by the CONDENSATION algorithm are then passed into the evaluation framework described in the preceding sections.

Overall, the evaluation will assess the two primary components of the system—Intelligent Scissors and the CONDENSATION tracking algorithm—using standard datasets and quantitative metrics. For segmentation, the Intelligent Scissors implementation will be tested on selected images from the PASCAL VOC 2012 dataset, with SSIM, FSIM, and MSE employed to measure structural similarity and pixel-level accuracy against processed ground truth annotations. Only the primary object per image will be considered to ensure consistency in comparisons. For object tracking, the CONDENSATION algorithm will be evaluated on a set of diverse video sequences, with Gaussian diffusion and translation parameters set beforehand according to the expected motion dynamics in each video. The goal of these evaluations will be to determine the effectiveness, accuracy, and robustness of the proposed system across varied visual and motion contexts.

7 Results and Discussions

7.1 Component 1: Performance of Intelligent Scissors

Results

To evaluate the performance of the implemented Intelligent Scissors segmentation approach, the output masks against ground truth annotations from 10 images of the PASCAL VOC 2012 dataset were compared.

Table 4: Segmentation Accuracy Metrics Across Test Images

Image ID	FSIM	SSIM	MSE
2007_000738	0.771822	0.999997	0.021217
2007_004143	0.794542	0.999998	0.014432
2007_004468	0.825060	0.999996	0.024713
2009_000285	0.739687	0.999997	0.016054
2009_003542	0.817549	0.999998	0.010511
2009_004620	0.823698	0.999998	0.016469
2010_001451	0.765410	0.999995	0.026274
2010_002047	0.826771	0.999999	0.009323
2010_004120	0.712238	0.999994	0.028059
2011_003030	0.762066	0.999998	0.015177
Average	**0.783884**	**0.999997**	**0.018223**

Discussion

The average SSIM of 0.999997 indicates that the segmented results preserve the structural content of the original images extremely well. This suggests the boundaries

selected by the Intelligent Scissors are well-aligned with annotated contours. The FSIM values range from 0.712 to 0.827, averaging 0.7839. These results demonstrate that while the method captures structural and perceptual details effectively, it may slightly underperform in cases with complex or low-contrast boundaries.
With an average MSE of 0.0182, the segmentation error is minimal. Lower MSE scores (e.g., 0.0093 for 2010_002047) suggest accurate edge alignment in many images, although some variability exists. These results confirm that the proposed framework in Section 4.2, involving a reduced feature set for Intelligent Scissors, provides quality boundary detection on natural images, particularly when strong edges are present. While SSIM and MSE metrics are consistently excellent, FSIM variation across images suggests some sensitivity to edge strength or local contrast.

7.2 Component 2: CONDENSATION Algorithm

Results

The following plots shows the center error for each video sequence, and the IoU across frames for every video sequence is described in the plots in Figures 16, 17, 18 and 19.

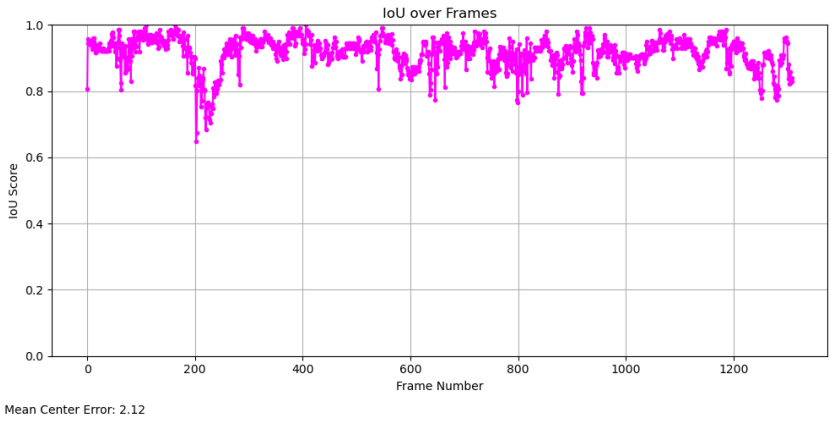


Figure 16: IoU over frames for TED Talk Speaker video sequence.

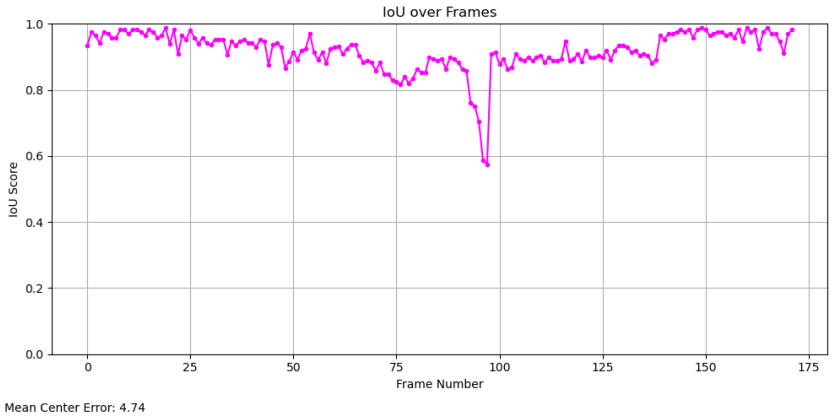


Figure 17: IoU over frames for Apple in Clutter video sequence.

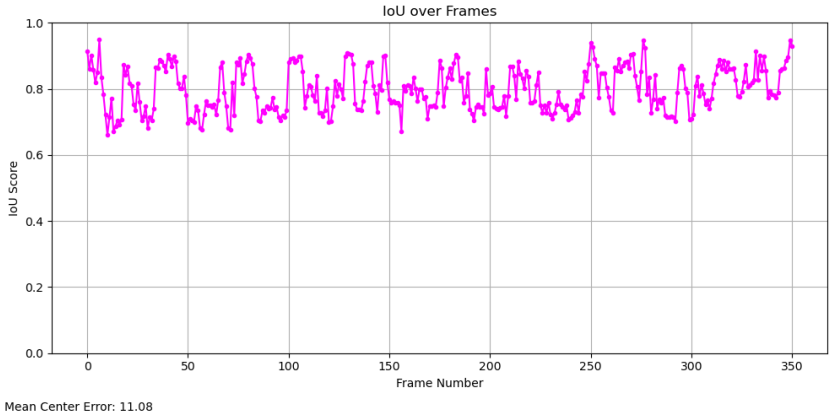


Figure 18: IoU over frames for Boat in Sea video sequence.

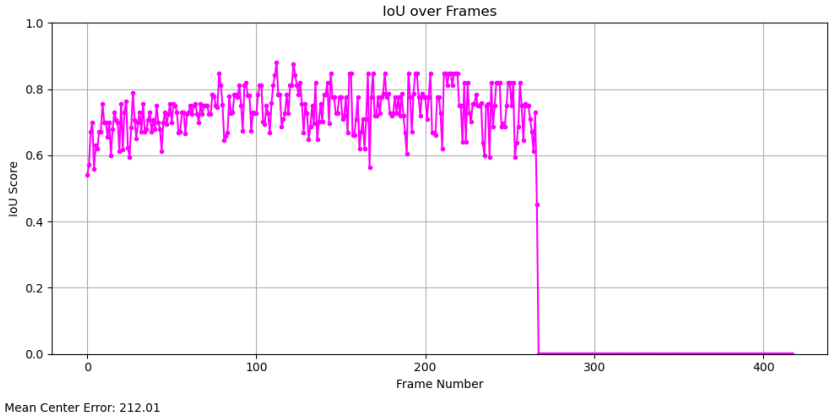


Figure 19: IoU over frames for Billiards video sequence.

Discussion

The proposed CONDENSATION-based contour tracking framework was evaluated across four diverse video sequences, each designed to test the system's performance under real-world variations while adhering to five core assumptions discussed in Section 2.2: gradual motion, rigid object structure, appearance consistency, two-dimensional translation, and offline tracking. Results demonstrate that the algorithm performs robustly and with high accuracy in scenarios that align with these constraints.

In the TED Talk sequence (Figure 16), where a speaker's face undergoes gradual movement with occasional head turns, the tracker achieved consistently high Intersection over Union (IoU) scores (0.85–1.0) and a low mean center error (2.12), indicating excellent stability and alignment. Despite minor angular deformations, the algorithm maintained its integrity, validating its suitability for tracking semi-rigid contours under predictable motion.

The apple-over-clutter sequence (Figure 17) further demonstrates the algorithm's resilience. Even when traversing a densely textured background, the IoU remained between 0.8 and 1.0, with only a brief dip to 0.6, quickly followed by a full recovery. This result demonstrates the algorithm's ability to correct and recover from temporary

misalignments, this can be attributed to the execution of practical considerations in the proposed framework in Section 4.3 – the use of fresh random samples to improve predictive ability. When the current sample set has samples that are too similar to each other and lack the actual object state, this demonstrated that the use of random samples can improve its predictive ability and recover from wrong predictions. This also shows the importance of setting an appropriate translation range for the random samples (Table 3); if the translation range is too high, the samples could overshoot predictions, and if too low, it would not be too different from the current sample set.

The boat sequence (Figure 18), characterised by environmental noise such as water textures and background motion, presented modest fluctuations in IoU (0.65–0.9) and a slightly elevated centre error (11.08). Nonetheless, the tracker maintained object alignment throughout, suggesting that the model is moderately robust to visual noise.

In contrast, the billiards sequence (Figure 19) revealed a key limitation. While tracking was initially stable, a sudden change in the ball's velocity caused a complete tracking failure, with IoU dropping to zero and a centre error of 212.01. This performance degradation shows the algorithm's sensitivity to abrupt motion changes, a direct consequence of its reliance on the gradual motion assumption highlighted in Section 2.2. This is a limitation present in the original CONDENSATION algorithm as well [13], as it uses the first order Markov process to model object dynamics, it assumes that motion changes are smooth and predictable from one frame to the next. Nonetheless, the accurate tracking of the small billiards ball throughout most of the video sequence is commendable, indicating that the algorithm demonstrates robustness to mild changes in velocity.

Overall, this implementation of the CONDENSATION algorithm demonstrates strong tracking performance when its foundational assumptions (Section 2.2) are satisfied. Its ability to recover from temporary failures and maintain contour alignment in cluttered environments is particularly encouraging. However, scenarios involving non-smooth motion or rapid object dynamics present challenges that warrant further development, such as incorporating adaptive motion models or enhanced observation likelihoods. These findings affirm this simplified variant of the CONDENSATION algorithm's application for offline tracking of rigid, gradually moving objects.

This chapter demonstrated that the Intelligent Scissors algorithm delivers accurate segmentation, with an average SSIM of 0.999997 and a low mean MSE of 0.0182, indicating strong structural alignment and minimal pixel-wise error compared to ground truth. FSIM scores, while slightly more variable, show effective feature-level similarity, affirming the algorithm's robustness in diverse boundary conditions. The CONDENSATION-based tracking framework performs reliably in controlled scenarios with gradual object motion, maintaining moderately good IoU scores and low center errors across most video sequences. However, its performance degrades during abrupt motion changes, as observed in the billiards video, which is a limitation of its underlying motion assumptions. Overall, the system proves effective for semi-automatic object segmentation and tracking in natural scenes when its core assumptions are met.

8 Conclusion

This dissertation aimed to develop and evaluate a bottom-up Segmentation-based Object Tracking (SOT) framework that integrates Intelligent Scissors for contour extraction and a simplified variant of the CONDENSATION algorithm for object tracking. The results obtained successfully demonstrate the feasibility and effectiveness of combining classical techniques with modern evaluation tools for real-world video object tracking. The research questions were addressed as follows:

1. **How can the object contour extracted by the Intelligent Scissors algorithm be effectively represented and incorporated within the CONDENSATION algorithm for robust contour-based tracking?**

   This work shows that the contour output from Intelligent Scissors can be effectively represented as a set of pixel coordinates, which can then be transformed using a translation-based state vector (Section 4.3) within a simplified CONDENSATION framework. This integration allowed for accurate and responsive tracking of the contour across frames with resilience to clutter and occlusion.

2. **What evaluation framework can be used to assess the tracking accuracy and contour consistency of the proposed framework across video sequences?**

   An evaluation framework was implemented using YOLOv11 object detections as surrogate ground truths, eliminating the need for manually annotated masks. The system computes IoU and centre error in real time, providing robust evaluation metrics to quantify tracking performance. The evaluation plots shown in Figures 16 to 19 closely reflect the observed tracking quality as seen by the human eye, quantifying the visual accuracy of the algorithm throughout each video sequence effectively.

3. **Can the proposed framework be implemented in a computationally efficient and user-friendly manner without compromising segmentation precision or tracking stability?**

   Yes. The Cython implementation of Intelligent Scissors reduced execution time by $\sim$97.5% for high resolution images, while the simplified CONDENSATION tracker operated at 3–5 FPS on modest hardware. The user interface remains intuitive, allowing for interactive seed placement and real-time feedback, thereby balancing efficiency and usability while compromising minimal output quality. The Intelligent Scissors algorithm maintained a robust average FSIM of 0.78, and a very low MSE value of 0.02, as shown in Table 4. The simplified CONDENSATION framework maintained robust IoU values over 0.70 for video sequences which did not violate the initial assumptions (Figures 16-18).

Together, these results demonstrate that the proposed framework achieves the aims and objectives set out in Section 2. The framework provides efficient, sufficiently accurate and interactive contour-based tracking, and can serve as a basis for future development in real-time vision applications.

9 Contributions

This dissertation presents several key contributions. A lightweight contour-based SOT framework is introduced, implemented and evaluated, combining the segmentation of Intelligent Scissors with the tracking of a simplified CONDENSATION tracking algorithm. Computational bottlenecks in the Intelligent Scissors module were overcome by re-implementing the path expansion logic in Cython, resulting in a ∼97.5% reduction in execution time. This optimisation enables responsive user interaction even on high-resolution images. The implementation of Intelligent Scissors achieves moderately good averages across all demonstrated evaluation metrics on a subset of the PASCAL VOC 2012 dataset.This dissertation also designed a simpler variant of the CONDENSATION algorithm which can be further developed and adapted for other applications. As demonstrated in the evaluation stage, it has similar characteristics (eg. robustness to clutter) as the original CONDENSATION algorithm [13], runs at 3-5 FPS and achieves moderately robust IoU scores (over 0.7) on diverse video sequences given the foundational constraints placed are not violated. An innovative evaluation approach was developed using YOLOv11 object detections to assess tracking performance. This eliminates the dependency on ground truth masks and enables evaluation in real-world videos, reflecting practical deployment conditions. Lastly, the software was tested on modest hardware, demonstrating its feasibility for edge deployment and interactive applications outside of high-performance environments. In addition, every component of the framework - Intelligent Scissors, CONDENSATION, and CNN-based evaluation - was intentionally designed with computational efficiency in mind, ensuring that the system could run on modest hardware. By considering computational efficiency, this framework is more accessible to resource-constrained applications as well.

10 Future Works

Lastly, there are many directions for future works. Limitations on the evaluation framework which need to be addressed include the YOLOv11 model's ability to only detect 80 classes, which means other object classes that need to be tracked would lack evaluation. Furthermore, this framework can be further developed for adaptive tracking by actively tuning the Gaussian diffusion and particle range parameters. Snake contours can be used on the rigid contour provided by Intelligent Scissors to allow for deformability of the object throughout the tracking process. Constraints and assumptions placed before the framework was created in Section 2.2 can also be addressed.

11 Reflection

11.1 Personal Reflection on the Plan

The initial work plan I prepared in December 2024 is as follows in Figure 20 below. The original plan expected the implementation of both CNN- and ViT-based evaluation frameworks by March 2025, with completion of the CONDENSATION algorithm scheduled for mid-January. However, the high computational demands of ViTs exceeded project constraints and led to their exclusion. Additionally, the CONDENSATION

algorithm required significantly more time than anticipated due to the steep learning curve and iterative refinement needed. These deviations highlighted the limitations of the original plan, which underestimated the complexity of certain components. This experience emphasised the importance of conducting a more thorough technical review before finalising project timelines.

TASK NAME	START DATE	END DATE	DURATION IN DAYS
Link Cost Calculations for Intelligent Scissors	09-28-24	10-12-24	14
Literature Review - Intelligent Scissors	08-01-24	10-29-24	89
Implementation of IS	08-01-24	10-13-24	73
Refactor Intelligent Scissors code - Jupyter Notebook to VS Code	10-29-24	11-29-24	31
Work on simple functional GUI for Intelligent Scissors	10-29-24	11-14-24	16
Literature Review - CONDENSATION Algorithm	11-01-24	11-15-24	14
Implementation of the CONDENSATION Algorithm	11-16-24	01-15-25	60
Literature Review - CNN/ViT	01-20-25	02-02-25	13
Implementation of CNN	01-23-25	02-28-25	36
Implementation of ViT	02-04-25	03-01-25	25
Evaluation	03-05-25	03-09-25	4
Final Report Writing	01-20-25	04-15-25	85

Figure 20: Initial work plan drafted in early December 2024.

As the project progressed, I realized that a rigid, deadline-driven approach was ill-suited to the evolving nature of research work. I adopted a more flexible strategy, including a detailed weekly progress log (Figure 21) to track tasks, adjust priorities, and reflect on progress. This shift improved efficiency and clarity across project stages. A key insight was the value of scheduling buffer time for unexpected challenges, especially with unfamiliar tools or algorithms. Writing the report alongside the project also led to a more accurate and detailed account. Moving forward, I'll plan with greater technical foresight, risk assessment, and flexible timelines grounded in realistic expectations.

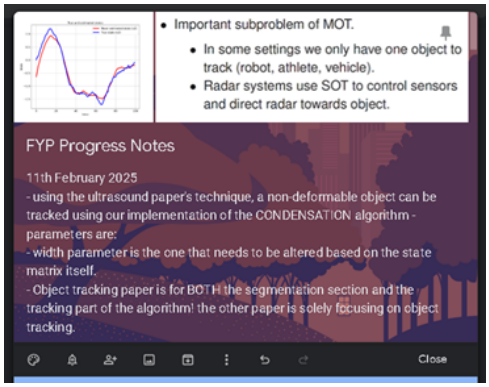


Figure 21: My progress notes which documented regular progress on a weekly basis.

11.2 Project Experience

Overall, this dissertation significantly challenged my technical and critical thinking skills, and completing it feels like a major personal achievement. One of the most demanding aspects was thoroughly understanding and implementing the CONDENSATION algorithm. It was a difficult learning curve - the original research paper by Isard et al. [13] was challenging to understand as it was heavy in notation and required a solid base in probability and statistics. I took initiative and started small – I took foundational courses online ("Intuitive Introduction to Probability" by Karl

Schmedders and "Introduction to Computer Vision" by Aaron Bobick) and kept re-reading the research paper to interpret it better, and iteratively re-worked through my implementations of the algorithm. I progressed slowly - from working with basic 2D shape tracking to handling diverse, real-world video sequences. It took me nearly two months to reach a point where I could confidently adapt and apply the algorithm on my own while still preserving the core framework, and seeing my implementation perform on real-life video sequences is a huge achievement for me. While the process was very challenging, it taught me the value of self-learning and how to interpret algorithms in research papers better.

Another key challenge was designing a custom GUI for the Intelligent Scissors tool, which required real-time hover detection, shortest-path calculation, and segment freezing—all while staying responsive. With few existing examples, I had to carefully interpret the desired user experience and translate it into an intuitive interface. Initially, implementing the tool in pure Python resulted in poor performance, taking up to 7 minutes for a single high-resolution image. This led me to discover and learn Cython, which allowed me to rewrite the tool for near-instantaneous performance. The process significantly sharpened my skills in performance optimisation and custom GUI development. Lastly, the evaluation phase required rigorous research. However after a rough scan through literature, my initial approach involved using the recent edge-detection model DexiNed [34] by Soria et al. to assess tracking accuracy, under the assumption that strong edge alignment would correlate with correct object tracking. I developed an entire evaluation framework based on this concept. However, during testing, I discovered that even when the CONDENSATION algorithm produced inaccurate tracking paths, the evaluation still reported high precision and recall scores. This inconsistency arose because the framework implicitly equated strong edge presence along the tracked contour with accurate tracking. In practice, the observation model was assigning higher probabilities to edge-rich regions, even when those regions did not correspond to the true object. As a result, the evaluation module failed to effectively penalise incorrect tracking behaviour. Recognising this flaw, I revised the evaluation strategy to better reflect tracking accuracy in a more object-aware manner by switching the framework to a state-of-art object detection model after a thorough literature review. This experience highlighted the importance of conducting a deeper literature review and critically assessing the validity of proposed evaluation methods before implementation.

In conclusion, this challenging project strengthened my confidence in tackling complex computational problems and enhanced my technical skills. It reshaped how I approach ambiguity—emphasising exploration, evaluation, and adaptation. I learned the value of patience, curiosity, and resilience, and gained the confidence to face unfamiliar challenges independently. These lessons will continue to guide my future work in research and software development.

11.3 Critical Appraisal

One of the key successes of this project was the ability to break down complex algorithms and rebuild them from first principles, particularly in the case of the CONDENSATION algorithm. The final implementation was moderately accurate and

adaptable, and I was able to apply it to real-world data with confidence. Another major achievement was the successful optimisation of the Intelligent Scissors tool, where learning and applying Cython significantly enhanced performance. Additionally, the custom GUI design met real-time interaction requirements, which was satisfying given the lack of similar precedents. However, there were also areas that did not go as smoothly. The initial evaluation framework, based on edge detection, proved to be a poor fit for the specific goals of tracking accuracy. This misstep consumed time and effort that could have been better spent had a more thorough literature review been conducted upfront. Similarly, the early stages of algorithm implementation were slow, largely due to underestimating the difficulty of interpreting academic research papers without adequate foundational knowledge. For future projects, dedicating more time early on to thorough background research and benchmarking will support better design choices. Adopting agile strategies—like iterative development, regular testing, and progress tracking—proved effective and should be implemented from the outset. Setting realistic timelines that account for learning curves and resource limits will also improve planning and reduce delays. Overall, this project was a valuable lesson in both technical execution and project management.

Bibliography

[1] R. Yao, G. Lin, S. Xia, J. Zhao, and Y. Zhou, 'Video object segmentation and tracking: A survey', *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 11, no. 4, pp. 1–47, 2020.

[2] A. Yilmaz, O. Javed, and M. Shah, 'Object tracking: A survey', *ACM Computing Surveys (CSUR)*, vol. 38, no. 4, pp. 13-es, 2006.

[3] V. Belagiannis, F. Schubert, N. Navab, and S. Ilic, 'Segmentation based particle filtering for real-time 2D object tracking', in *Computer Vision–ECCV 2012*, Florence, Italy, Oct. 7–13, 2012, pp. 842–855.

[4] J. Son, I. Jung, K. Park, and B. Han, 'Tracking-by-segmentation with online gradient boosting decision tree', in *Proc. IEEE Int. Conf. Computer Vision*, 2015, pp. 3056–3064.

[5] E. N. Mortensen and W. A. Barrett, 'Intelligent scissors for image composition', in *Proc. 22nd Annual Conf. Computer Graphics and Interactive Techniques*, 1995, pp. 191–198.

[6] E. N. Mortensen and W. A. Barrett, 'Toboggan-based intelligent scissors with a four-parameter edge model', in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 1999, vol. 2, pp. 452–458.

[7] W. A. Barrett, L. J. Reese, and E. N. Mortensen, 'Intelligent segmentation tools', in *Proc. IEEE International Symposium on Biomedical Imaging*, 2002, pp. 217–220.

[8] D. Farin, M. Pfeffer, P. H. N. de With, and W. Effelsberg, 'Corridor scissors: A semiautomatic segmentation tool employing minimum-cost circular paths', in *Proc. Int. Conf. Image Processing (ICIP)*, 2004, vol. 2, pp. 1177–1180.

[9] D. Stalling and H.-C. Hege, 'Intelligent scissors for medical image segmentation', in *Proc. 4th Freiburger Workshop Digitale Bildverarbeitung in der Medizin*, Freiburg, 1996, pp. 32–36.

[10] J. Jin and X. Tong, 'An improved algorithm for interactive medical image segmentation based on intelligent scissors', in *Proc. 4th Int. Conf. on Computer Vision, Image and Deep Learning (CVIDL)*, 2023, pp. 175–179.

[11] Y. Gaobo and Y. Shengfa, 'Modified intelligent scissors and adaptive frame skipping for video object segmentation', *Real-Time Imaging*, vol. 11, no. 4, pp. 310–322, 2005.

[12] Y. Li, H. Pan, L. Jiang, J. Tong, and Y. Shen, 'An improved intelligent scissors algorithm for the segmentation of vessels segments in coronary angiography imaging', in *Proc. 3rd Int. Conf. Intelligent Robotic and Control Engineering (IRCE)*, 2020, pp. 62–67.

[13] M. Isard and A. Blake, 'CONDENSATION—conditional density propagation for visual tracking', *International Journal of Computer Vision*, vol. 29, no. 1, pp. 5–28, 1998.

[14] Y. M. Lui, J. R. Beveridge, and L. D. Whitley, 'Adaptive appearance model and condensation algorithm for robust face tracking', *IEEE Trans. Systems, Man, and Cybernetics-Part A*, vol. 40, no. 3, pp. 437–448, 2010.

[15] X. Zhang, M. Günther, and A. Bongers, 'Real-time organ tracking in ultrasound imaging using active contours and conditional density propagation', in *Medical Imaging and Virtual Reality*, 2010, pp. 286–294.

[16] A. M. Hafiz and G. M. Bhat, 'A survey on instance segmentation: state of the art', *International Journal of Multimedia Information Retrieval*, vol. 9, no. 3, pp. 171–189, 2020.

[17] R. Sharma, M. Saqib, C.-T. Lin, and M. Blumenstein, 'A survey on object instance segmentation', *SN Computer Science*, vol. 3, no. 6, p. 499, 2022.

[18] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, 'Object detection in 20 years: A survey', *Proceedings of the IEEE*, vol. 111, no. 3, pp. 257–276, 2023.

[19] A. Dosovitskiy et al., 'An image is worth 16x16 words: Transformers for image recognition at scale', *arXiv preprint* arXiv:2010.11929, 2020.

[20] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, 'End-to-end object detection with transformers', in *European Conference on Computer Vision*, 2020, pp. 213–229.

[21] A. B. Amjoud and M. Amrouch, 'Object detection using deep learning, CNNs and vision transformers: A review', *IEEE Access*, vol. 11, pp. 35479–35516, 2023.

[22] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, 'Deformable DETR: Deformable transformers for end-to-end object detection', *arXiv preprint* arXiv:2010.04159, 2020.

[23] H. Zhang et al., 'DINO: DETR with improved denoising anchor boxes for end-to-end object detection', *arXiv preprint* arXiv:2203.03605, 2022.

[24] Y. Zhao et al., 'DETRs beat YOLOs on real-time object detection', in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition*, 2024, pp. 16965–16974.

[25] M. Tan, R. Pang, and Q. V. Le, 'EfficientDet: Scalable and efficient object detection', in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition*, 2020, pp. 10781–10790.

[26] N. Jegham, C. Y. Koh, M. Abdelatti, and A. Hendawi, 'YOLO Evolution: A Comprehensive Benchmark and Architectural Review of YOLOv12, YOLO11, and Their Previous Versions', *YOLO11 Technical Report*.

[27] S. Ren, K. He, R. Girshick, and J. Sun, 'Faster R-CNN: Towards real-time object detection with region proposal networks', *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[28] Ultralytics, 'RTDETRv2 vs YOLO11: A technical comparison for object detection', May 1, 2024. [Online]. Available: `https://docs.ultralytics.com/compare/rtdetr-vs-yolo11/`. [Accessed: May 2, 2025].

[29] E. N. Mortensen and W. A. Barrett, 'Interactive segmentation with intelligent scissors', *Graphical Models and Image Processing*, vol. 60, no. 5, pp. 349–384, 1998.

[30] N. Vaswani, Y. Rathi, A. Yezzi, and A. Tannenbaum, 'Deform PF-MT: Particle filter with mode tracker for tracking nonaffine contour deformations', *IEEE Trans. Image Processing*, vol. 19, no. 4, pp. 841–857, 2009.

[31] R. Khanam and M. Hussain, 'YOLOv11: An overview of the key architectural enhancements', *arXiv preprint* arXiv:2410.17725, 2024.

[32] Z. Soleimanitaleb and M. A. Keyvanrad, 'Single object tracking: A survey of methods, datasets, and evaluation metrics', *arXiv preprint* arXiv:2201.13066, 2022.

[33] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, 'The Pascal Visual Object Classes (VOC) Challenge', *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, Jun. 2010.

[34] X. Soria, A. Sappa, P. Humanante, and A. Akbarinia, 'Dense extreme inception network for edge detection', *Pattern Recognition*, vol. 139, p. 109461, 2023.